Automated Validation / Acceptance Testing

The Short Version

`scripts/validate_drift.py` automatically checks the pipeline against real, labeled data and prints a pass/fail verdict — replacing what used to be a handful of manual database checks run by hand after every change. It's a genuine, useful piece of automation, and it's worth being precise about exactly what it does and doesn't cover.

What Used to Be Manual

Before this script existed, verifying the pipeline still worked after a change meant a sequence of manual steps: check row counts didn't drop, run a few test queries and eyeball whether the results looked sensible, check the cohort breakdown, run a known drift-style query and confirm drift rows actually won. Each check was useful; doing them by hand, every time, was slow and easy to skip under time pressure — which is exactly how a real regression slips through unnoticed.

What the Script Actually Checks

Three SQL-backed checks, run in sequence:

1. **A baseline exists** — at least one route has a computed baseline centroid, confirming the detector's setup actually completed.
2. **Something has been flagged** — at least one row exists in `drift_events`, confirming the detector is running at all.
3. **The flagged sessions are genuinely correct** — at least one flagged session actually carries the ground-truth `cohort = 'drift'` label, not just any flag. This is the check that matters: it's not enough for the detector to flag *something*, it has to be flagging the right thing, and that's only checkable because of the labeled synthetic dataset (see the Synthetic Dataset page).

If all three pass, the script exits cleanly with a pass message. If any fail, it names exactly which one and exits with an error — which means this could be wired into CI to block a bad change automatically, not just run manually on demand.

What "Zero Manual Steps" Actually Means — and Doesn't

It's accurate to say this eliminated manual steps for verifying **detection correctness** specifically. It is not accurate to say the entire system requires zero manual verification, and the script itself is explicit about this rather than overselling it.

Of the project's three acceptance criteria, only one — true-positive drift detection — is automated by this script. The other two still require a real manual check:

- **The dashboard boots cleanly from a fresh `docker-compose up`** — still needs an actual manual run.
- **A reviewer can navigate the dashboard unaided** — inherently a human judgment call, not something a script can verify.

The script prints this limitation every time it runs, specifically so a passing result is never mistaken for "everything is verified."

Why the Honest Framing Matters

An automated check that quietly overstates its own coverage is worse than no automation at all — it creates false confidence. Stating plainly what is and isn't covered is what makes "zero manual steps" a claim worth trusting, rather than a claim worth double-checking.